

Plano de Implementação Revisado do MVP

Sistema de Gestão Financeira para Igrejas

Versão: Revisada para MVP produtivo

Data: 14/05/2026

Objetivo: transformar o escopo inicial em um plano técnico executável, seguro, enxuto e preparado para evolução como SaaS.

1. Resumo Executivo

O Sistema de Gestão Financeira para Igrejas será uma aplicação web para controle financeiro de instituições religiosas, permitindo o registro de entradas, saídas, categorias, dashboard financeiro e relatórios por período com exportação em PDF e Excel.

A versão MVP não deve tentar resolver todos os problemas do produto final. O objetivo da primeira entrega é validar o núcleo operacional do sistema:

- cadastrar uma igreja;
- autenticar usuários;
- registrar dízimos e ofertas;
- registrar despesas por categoria;
- acompanhar o saldo financeiro;
- gerar relatórios simples e confiáveis;
- garantir segurança, separação de dados e rastreabilidade básica.

A implementação deve ser feita com arquitetura simples, código limpo, validação no backend, proteção contra falhas comuns e banco preparado desde o início para múltiplas igrejas.

2. Direção Técnica do MVP

2.1 Decisão principal

O sistema deve nascer preparado para multi-igreja, mesmo que a primeira versão seja usada por apenas uma instituição.

Isso significa que as tabelas principais devem conter `igreja_id` desde o início. Não fazer isso agora deixaria o MVP aparentemente mais simples, mas causaria retrabalho grande quando o produto evoluir para atender várias igrejas.

2.2 Stack recomendada

Camada	Tecnologia recomendada
Backend	PHP 8.4+ preferencialmente; PHP 8.2+ se a hospedagem limitar
Banco de dados	MySQL 8.4 LTS preferencialmente; MySQL 8+ se a hospedagem limitar
Frontend	HTML5, CSS3 e JavaScript Vanilla
Dependências	Composer
PDF	mPDF ou TCPDF
Excel	PhpSpreadsheet
Servidor	Apache ou Nginx
Arquitetura	MVC leve, sem framework pesado
Autenticação	Sessão segura com cookies protegidos
Banco	PDO com prepared statements

2.3 O que não usar no MVP

Não usar JWT como autenticação principal no MVP web tradicional. Para uma aplicação PHP renderizada no servidor, sessão segura é mais adequada, mais simples e mais fácil de proteger.

JWT só deve entrar futuramente se houver:

- aplicativo mobile;
- API pública;
- frontend separado em SPA;
- integração externa.

3. Escopo Revisado do MVP

3.1 Incluído no MVP

Módulo	Funcionalidades
Igrejas	Cadastro base da igreja, status e identificação interna
Usuários	Cadastro de usuário administrador inicial, login, logout e edição básica de perfil
Autenticação	Sessão segura, proteção de rotas privadas, timeout e CSRF
Papéis simples	admin, tesoureiro e visualizador
Categorias	Categorias padrão, criação, edição e desativação

Módulo	Funcionalidades
Entradas	Registro, listagem, edição, exclusão, filtro e paginação de dígitos/ofertas
Saídas	Registro, listagem, edição, exclusão, filtro e paginação de despesas
Dashboard	Cards financeiros, saldo, totais do mês e últimas movimentações
Relatórios	Resumo financeiro por período, PDF simples e Excel simples
Auditoria mínima	Registro de criação, edição e exclusão de movimentações financeiras
Segurança	Validação backend, PDO, CSRF, escaping HTML, controle por <code>igreja_id</code>
Responsividade	Layout funcional para celular, tablet e desktop

3.2 Fora do MVP inicial

Funcionalidade	Motivo para ficar fora
App mobile	Aumenta custo e complexidade antes da validação do núcleo financeiro
Integração bancária	Exige conciliação, APIs externas, tratamento de falhas e regras específicas
Pagamentos online	Pode virar outro produto dentro do produto
Relatórios agendados por email	Exige cron, fila, SMTP confiável, logs e reprocessamento
Permissões avançadas	No MVP, papéis simples são suficientes
Análise preditiva	Não é necessária para validar o controle financeiro básico
Integração contábil	Deve vir depois da validação do uso real
Aplicativo PWA avançado	Pode entrar como evolução após o MVP web validado
Backup automático avançado	No MVP, iniciar com política simples e documentação operacional

4. Regras de Negócio do MVP

4.1 Igrejas

- Cada igreja deve ter seus próprios usuários, categorias, entradas, saídas e relatórios.
- Nenhuma igreja pode visualizar, editar, excluir ou exportar dados de outra igreja.
- Toda consulta financeira deve obrigatoriamente filtrar por `igreja_id`.

4.2 Usuários

- O primeiro usuário criado para uma igreja será administrador.

- O usuário administrador pode gerenciar categorias, entradas, saídas e relatórios.
- O tesoureiro pode gerenciar movimentações financeiras.
- O visualizador pode apenas consultar dashboard e relatórios.
- Todo usuário pertence a uma única igreja no MVP.

4.3 Entradas

- Entrada pode ser do tipo `dizimo` ou `oferta`.
- O valor deve ser positivo.
- A data não deve ser futura, salvo se houver permissão futura específica.
- A descrição é opcional, mas deve ter limite de caracteres.
- A entrada deve registrar quem criou e quem alterou.

4.4 Saídas

- Saída deve pertencer a uma categoria ativa.
- O valor deve ser positivo.
- A data não deve ser futura no MVP.
- A descrição é opcional, mas deve ter limite de caracteres.
- A saída deve registrar quem criou e quem alterou.

4.5 Categorias

- Categorias pertencem a uma igreja.
- O nome da categoria deve ser único dentro da mesma igreja.
- Categorias usadas em saídas não devem ser removidas fisicamente; devem ser desativadas.

4.6 Relatórios

- Relatórios devem ser gerados por período.
- O sistema deve calcular:
 - total de dízimos;
 - total de ofertas;
 - total de entradas;
 - total de saídas;
 - saldo;
 - saídas por categoria.
- O relatório deve considerar apenas dados da igreja logada.

5. Arquitetura Recomendada

A arquitetura deve ser simples, organizada e pronta para produção. O objetivo é evitar tanto um sistema desorganizado em arquivos soltos quanto uma estrutura pesada demais para o MVP.

5.1 Estrutura de diretórios

```
app/  
  Controllers/  
    AuthController.php  
    DashboardController.php  
    EntradaController.php  
    SaidaController.php  
    CategoriaController.php  
    RelatorioController.php  
    ConfiguracaoController.php  
  
  Models/  
    Igreja.php  
    Usuario.php  
    Categoria.php  
    Entrada.php  
    Saida.php  
    Auditoria.php  
  
  Core/  
    Router.php  
    Database.php  
    View.php  
    Validator.php  
    Response.php  
    Session.php  
  
  Middleware/  
    AuthMiddleware.php  
    RoleMiddleware.php  
    CsrfMiddleware.php  
    TenantMiddleware.php  
  
  Services/  
    AuthService.php  
    AuditoriaService.php  
    RelatorioService.php  
    ExportPdfService.php  
    ExportExcelService.php  
  
config/  
  app.php  
  database.php  
  security.php  
  
views/  
  layouts/  
    app.php  
    auth.php
```

```

auth/
  login.php
  register.php
dashboard/
  index.php
entradas/
  index.php
  form.php
saidas/
  index.php
  form.php
categorias/
  index.php
relatorios/
  index.php
configuracoes/
  index.php

public/
  index.php
  assets/
    css/
      app.css
    js/
      app.js
    img/

database/
  migrations/
  seeds/
  init.sql

storage/
  logs/
  exports/

vendor/
.env.example
composer.json
README.md

```

5.2 Separação de responsabilidades

Camada	Responsabilidade
Controller	Receber requisição, acionar validações e chamar serviços/modelos
Model	Representar entidades e operações de banco
Service	Regras de negócio mais complexas, relatórios, auditoria e exportações

Camada	Responsabilidade
Middleware	Autenticação, autorização, CSRF e isolamento por igreja
View	Renderização HTML segura
Core	Infraestrutura da aplicação: rotas, conexão, sessão e validação

6. Modelo de Dados Revisado

6.1 Tabela `igrejas`

```
CREATE TABLE igrejas (
  id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  nome VARCHAR(180) NOT NULL,
  cnpj VARCHAR(20) NULL,
  email VARCHAR(180) NULL,
  telefone VARCHAR(30) NULL,
  status ENUM('ativa', 'inativa') NOT NULL DEFAULT 'ativa',
  criado_em TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  atualizado_em TIMESTAMP NULL DEFAULT NULL ON UPDATE CURRENT_TIMESTAMP,
  INDEX idx_igrejas_status (status)
);
```

6.2 Tabela `usuarios`

```
CREATE TABLE usuarios (
  id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  igreja_id BIGINT UNSIGNED NOT NULL,
  nome VARCHAR(180) NOT NULL,
  email VARCHAR(180) NOT NULL,
  senha_hash VARCHAR(255) NOT NULL,
  papel ENUM('admin', 'tesoureiro', 'visualizador') NOT NULL DEFAULT
  'admin',
  ativo TINYINT(1) NOT NULL DEFAULT 1,
  ultimo_login_em TIMESTAMP NULL,
  criado_em TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  atualizado_em TIMESTAMP NULL DEFAULT NULL ON UPDATE CURRENT_TIMESTAMP,

  UNIQUE KEY uk_usuarios_igreja_email (igreja_id, email),
  INDEX idx_usuarios_igreja (igreja_id),
  INDEX idx_usuarios_ativo (ativo),

  CONSTRAINT fk_usuarios_igreja
    FOREIGN KEY (igreja_id) REFERENCES igrejas(id)
```

```
        ON DELETE RESTRICT
    );
```

6.3 Tabela categorias

```
CREATE TABLE categorias (
    id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    igreja_id BIGINT UNSIGNED NOT NULL,
    nome VARCHAR(120) NOT NULL,
    descricao TEXT NULL,
    cor VARCHAR(7) NOT NULL DEFAULT '#E8D4F8',
    ativo TINYINT(1) NOT NULL DEFAULT 1,
    criado_em TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    atualizado_em TIMESTAMP NULL DEFAULT NULL ON UPDATE CURRENT_TIMESTAMP,

    UNIQUE KEY uk_categorias_igreja_nome (igreja_id, nome),
    INDEX idx_categorias_igreja_ativo (igreja_id, ativo),

    CONSTRAINT fk_categorias_igreja
        FOREIGN KEY (igreja_id) REFERENCES igrejas(id)
        ON DELETE RESTRICT
);
```

6.4 Tabela entradas

```
CREATE TABLE entradas (
    id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    igreja_id BIGINT UNSIGNED NOT NULL,
    usuario_id BIGINT UNSIGNED NOT NULL,
    tipo ENUM('dizimo', 'oferta') NOT NULL,
    valor DECIMAL(12,2) NOT NULL,
    descricao TEXT NULL,
    contribuinte_nome VARCHAR(180) NULL,
    forma_pagamento ENUM('dinheiro', 'pix', 'cartao', 'transferencia',
        'outro') NULL,
    data_entrada DATE NOT NULL,
    criado_em TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    atualizado_em TIMESTAMP NULL DEFAULT NULL ON UPDATE CURRENT_TIMESTAMP,

    INDEX idx_entradas_igreja_data (igreja_id, data_entrada),
    INDEX idx_entradas_igreja_tipo (igreja_id, tipo),
    INDEX idx_entradas_usuario (usuario_id),

    CONSTRAINT fk_entradas_igreja
        FOREIGN KEY (igreja_id) REFERENCES igrejas(id)
        ON DELETE RESTRICT,

    CONSTRAINT fk_entradas_usuario
```



```

        FOREIGN KEY (usuario_id) REFERENCES usuarios(id)
        ON DELETE RESTRICT,

    CONSTRAINT chk_entradas_valor_positivo
        CHECK (valor > 0)
);

```

6.5 Tabela `saidas`

```

CREATE TABLE saidas (
    id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    igreja_id BIGINT UNSIGNED NOT NULL,
    usuario_id BIGINT UNSIGNED NOT NULL,
    categoria_id BIGINT UNSIGNED NOT NULL,
    valor DECIMAL(12,2) NOT NULL,
    descricao TEXT NULL,
    fornecedor VARCHAR(180) NULL,
    forma_pagamento ENUM('dinheiro', 'pix', 'cartao', 'transferencia',
        'boleto', 'outro') NULL,
    data_saida DATE NOT NULL,
    criado_em TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    atualizado_em TIMESTAMP NULL DEFAULT NULL ON UPDATE CURRENT_TIMESTAMP,

    INDEX idx_saidas_igreja_data (igreja_id, data_saida),
    INDEX idx_saidas_igreja_categoria (igreja_id, categoria_id),
    INDEX idx_saidas_usuario (usuario_id),

    CONSTRAINT fk_saidas_igreja
        FOREIGN KEY (igreja_id) REFERENCES igrejas(id)
        ON DELETE RESTRICT,

    CONSTRAINT fk_saidas_usuario
        FOREIGN KEY (usuario_id) REFERENCES usuarios(id)
        ON DELETE RESTRICT,

    CONSTRAINT fk_saidas_categoria
        FOREIGN KEY (categoria_id) REFERENCES categorias(id)
        ON DELETE RESTRICT,

    CONSTRAINT chk_saidas_valor_positivo
        CHECK (valor > 0)
);

```

6.6 Tabela `logs_auditoria`

```

CREATE TABLE logs_auditoria (
    id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    igreja_id BIGINT UNSIGNED NOT NULL,

```

```

usuario_id BIGINT UNSIGNED NULL,
acao VARCHAR(80) NOT NULL,
tabela_afetada VARCHAR(80) NOT NULL,
registro_id BIGINT UNSIGNED NULL,
dados_anteriores JSON NULL,
dados_novos JSON NULL,
ip VARCHAR(45) NULL,
user_agent VARCHAR(255) NULL,
criado_em TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,

INDEX idx_logs_igreja_data (igreja_id, criado_em),
INDEX idx_logs_usuario (usuario_id),

CONSTRAINT fk_logs_igreja
    FOREIGN KEY (igreja_id) REFERENCES iglesias(id)
    ON DELETE RESTRICT,

CONSTRAINT fk_logs_usuario
    FOREIGN KEY (usuario_id) REFERENCES usuarios(id)
    ON DELETE SET NULL
);

```

7. Segurança Obrigatória

7.1 Autenticação

- Usar `password_hash()` para salvar senhas.
- Usar `password_verify()` para validar login.
- Nunca salvar senha em texto puro.
- Regenerar o ID da sessão após login com `session_regenerate_id(true)`.
- Implementar logout destruindo a sessão corretamente.
- Implementar timeout por inatividade.
- Mensagens de erro de login devem ser genéricas.

Exemplo de mensagem correta:

Email ou senha inválidos.

Não usar mensagens como:

Email não encontrado.

Isso evita enumeração de usuários.

7.2 Sessão

Cookies de sessão devem usar:

- `HttpOnly`;
- `Secure` em produção;
- `SameSite=Lax` ou `SameSite=Strict`;
- tempo de expiração controlado.

7.3 CSRF

Todo formulário que altera dados deve ter token CSRF:

- login;
- registro;
- criação de entrada;
- edição de entrada;
- exclusão de entrada;
- criação de saída;
- edição de saída;
- exclusão de saída;
- categorias;
- configurações.

7.4 Autorização por igreja

Toda query financeira deve ter `igreja_id`.

Errado:

```
SELECT * FROM entradas WHERE id = :id;
```

Correto:

```
SELECT * FROM entradas  
WHERE id = :id  
AND igreja_id = :igreja_id;
```

Essa regra é obrigatória para visualizar, editar, excluir e exportar dados.

7.5 Proteção contra XSS

Toda saída HTML deve ser escapada:

```
htmlspecialchars($valor, ENT_QUOTES, 'UTF-8');
```

7.6 SQL Injection

Toda operação de banco deve usar PDO com prepared statements.

Não concatenar dados do usuário em SQL.

7.7 Rate limit de login

Implementar proteção simples contra força bruta:

- limitar tentativas por IP e email;
- aplicar bloqueio temporário após várias falhas;
- registrar tentativas suspeitas.

7.8 Tratamento de erros

Em produção:

- não exibir stack trace;
- não exibir SQL;
- não exibir caminho interno de arquivos;
- registrar erro em log;
- mostrar mensagem amigável ao usuário.

8. Interface e UX do MVP

A interface deve ser moderna, limpa, responsiva e simples. O foco do MVP não é animação avançada; é clareza operacional.

8.1 Páginas obrigatórias

Página	Rota	Objetivo
Login	<code>/login</code>	Acesso seguro ao sistema
Registro inicial	<code>/registro</code>	Criar igreja e usuário administrador inicial
Dashboard	<code>/dashboard</code>	Visão geral financeira
Entradas	<code>/entradas</code>	Gestão de dízimos e ofertas
Saídas	<code>/saidas</code>	Gestão de despesas
Categorias	<code>/categorias</code>	Gestão das categorias de despesas
Relatórios	<code>/relatorios</code>	Resumo e exportações
Configurações	<code>/configuracoes</code>	Perfil, senha e dados básicos da igreja

8.2 Dashboard

O dashboard deve exibir:

- total de entradas do mês;
- total de dígitos do mês;
- total de ofertas do mês;
- total de saídas do mês;
- saldo do mês;
- últimas movimentações;
- atalhos para nova entrada, nova saída e relatório.

8.3 Entradas

A tela de entradas deve conter:

- botão de nova entrada;
- tipo: dígito ou oferta;
- valor;
- data;
- forma de pagamento;
- contribuinte opcional;
- descrição;
- filtros por tipo e período;
- paginação;
- totalizador do filtro atual;
- ações de editar e excluir.

8.4 Saídas

A tela de saídas deve conter:

- botão de nova saída;
- categoria;
- valor;
- data;
- forma de pagamento;
- fornecedor opcional;
- descrição;
- filtros por categoria e período;
- paginação;
- totalizador do filtro atual;
- ações de editar e excluir.

8.5 Relatórios

A tela de relatórios deve conter:

- data inicial;
- data final;

- botão para visualizar resumo;
 - botão para exportar PDF;
 - botão para exportar Excel;
 - resumo na tela com totais principais;
 - tabela de saídas por categoria.
-

9. Fases de Implementação

Fase 0 — Preparação do projeto

Entregáveis

- Estrutura de pastas.
- Composer configurado.
- `.env.example`.
- Conexão PDO.
- Router básico.
- Página inicial.
- Tratamento inicial de erros.

Critérios de aceite

- Aplicação sobe localmente.
 - Conecta ao banco.
 - Renderiza página inicial sem erro.
 - `.env` real não está versionado.
-

Fase 1 — Banco de dados e seeds

Entregáveis

- Migration ou script SQL de:
 - igrejas;
 - usuários;
 - categorias;
 - entradas;
 - saídas;
 - logs de auditoria.
- Seeds de categorias padrão.
- Índices e foreign keys.

Critérios de aceite

- Banco pode ser recriado do zero por script.
 - Constraints impedem dados básicos inválidos.
 - Categorias padrão são criadas para a igreja inicial.
-

Fase 2 — Autenticação e segurança base

Entregáveis

- Registro inicial de igreja e admin.
- Login.
- Logout.
- Sessão segura.
- Middleware de autenticação.
- CSRF.
- Timeout de sessão.

Critérios de aceite

- Usuário cria conta e faz login.
 - Usuário faz logout.
 - Rotas privadas bloqueiam acesso sem sessão.
 - Token CSRF bloqueia requisições inválidas.
-

Fase 3 — Layout base responsivo

Entregáveis

- Layout principal.
- Sidebar ou menu responsivo.
- Header com usuário logado.
- Componentes base:
 - cards;
 - tabelas;
 - botões;
 - inputs;
 - modais;
 - toasts;
 - estados vazios.

Critérios de aceite

- Interface funciona em celular, tablet e desktop.
 - Menu é usável no mobile.
 - Contraste e leitura estão adequados.
-

Fase 4 — Categorias

Entregáveis

- Listar categorias.
- Criar categoria.
- Editar categoria.

- Desativar categoria.
- Validar cor hexadecimal.
- Impedir nome duplicado dentro da mesma igreja.

Critérios de aceite

- Categoria criada aparece nas saídas.
 - Categoria desativada não aparece para novos lançamentos.
 - Categoria usada em saída não é excluída fisicamente.
-

Fase 5 — Entradas

Entregáveis

- CRUD de dízimos e ofertas.
- Filtro por tipo.
- Filtro por período.
- Paginação.
- Totalizador.
- Auditoria em criação, edição e exclusão.

Critérios de aceite

- Usuário cria, edita, lista, filtra e exclui entradas.
 - Usuário só acessa entradas da própria igreja.
 - Valores inválidos são bloqueados no backend.
-

Fase 6 — Saídas

Entregáveis

- CRUD de despesas.
- Seleção de categoria.
- Filtro por categoria.
- Filtro por período.
- Paginação.
- Totalizador.
- Auditoria em criação, edição e exclusão.

Critérios de aceite

- Usuário cria, edita, lista, filtra e exclui saídas.
 - Usuário só acessa saídas da própria igreja.
 - Categoria inválida ou inativa é bloqueada.
-

Fase 7 — Dashboard

Entregáveis

- Cards financeiros.
- Totais do mês.
- Saldo do período.
- Últimas movimentações.
- Ações rápidas.

Critérios de aceite

- Dashboard reflete os dados reais cadastrados.
 - Ao registrar entrada ou saída, os totais atualizam corretamente.
 - Valores são calculados apenas pela igreja logada.
-

Fase 8 — Relatórios e exportações

Entregáveis

- Relatório por período.
- Resumo financeiro.
- Saídas por categoria.
- Exportação PDF.
- Exportação Excel.

Critérios de aceite

- Relatório mostra dados corretos por período.
 - PDF baixa corretamente.
 - Excel baixa corretamente.
 - Arquivos não misturam dados de outras igrejas.
-

Fase 9 — Configurações

Entregáveis

- Editar nome do usuário.
- Editar email do usuário.
- Alterar senha.
- Editar dados básicos da igreja.

Critérios de aceite

- Alteração de senha exige senha atual.
 - Email novo é validado.
 - Dados da igreja são atualizados apenas por admin.
-

Fase 10 — Hardening e homologação

Entregáveis

- Revisão de segurança.
- Testes manuais de fluxo crítico.
- Testes de responsividade.
- Revisão de performance.
- Configuração de produção.
- Documentação de instalação.

Critérios de aceite

- Nenhuma rota sensível fica exposta.
- Nenhum erro interno aparece para o usuário.
- Sistema funciona em ambiente limpo.
- MVP está pronto para homologação.

10. Backlog Técnico Priorizado

P0 — Obrigatório para lançar o MVP

Código	Tarefa	Módulo
P0-001	Criar estrutura MVC do projeto	Infra
P0-002	Configurar Composer e autoload	Infra
P0-003	Criar <code>.env.example</code>	Infra
P0-004	Criar conexão PDO segura	Infra
P0-005	Criar router básico	Infra
P0-006	Criar migrations/scripts SQL	Banco
P0-007	Criar seeds de categorias padrão	Banco
P0-008	Implementar registro inicial de igreja/admin	Auth
P0-009	Implementar login com <code>password_verify</code>	Auth
P0-010	Implementar logout seguro	Auth
P0-011	Implementar middleware de autenticação	Segurança
P0-012	Implementar proteção CSRF	Segurança
P0-013	Implementar controle por <code>igreja_id</code>	Segurança
P0-014	Criar layout base responsivo	UI
P0-015	Criar CRUD de categorias	Categorias

Código	Tarefa	Módulo
P0-016	Criar CRUD de entradas	Entradas
P0-017	Criar CRUD de saídas	Saídas
P0-018	Criar dashboard financeiro	Dashboard
P0-019	Criar relatório por período	Relatórios
P0-020	Exportar relatório em PDF	Relatórios
P0-021	Exportar relatório em Excel	Relatórios
P0-022	Criar logs de auditoria	Auditoria
P0-023	Testar rotas protegidas	QA
P0-024	Testar isolamento entre igrejas	QA
P0-025	Preparar deploy de homologação	Deploy

P1 — Depois do MVP validado

Código	Tarefa	Módulo
P1-001	Recuperação de senha por email	Auth
P1-002	Convite de novos usuários	Usuários
P1-003	Permissões mais granulares	Segurança
P1-004	Relatórios com gráficos avançados	Relatórios
P1-005	Backup automático	Infra
P1-006	Busca avançada de movimentações	Financeiro
P1-007	Comparativo mensal no dashboard	Dashboard
P1-008	Histórico visual de auditoria	Auditoria

P2 — Roadmap comercial

Código	Tarefa	Módulo
P2-001	Planos e assinatura	SaaS
P2-002	Integração bancária	Financeiro
P2-003	Conciliação financeira	Financeiro
P2-004	Relatórios agendados por email	Relatórios
P2-005	PWA ou app mobile	Mobile
P2-006	Integração contábil	Contabilidade
P2-007	Análise preditiva	Inteligência

11. Cronograma Recomendado

O cronograma assume um desenvolvedor full stack trabalhando com escopo controlado, sem mudanças grandes durante o ciclo.

Semana	Foco	Resultado esperado
Semana 1	Base técnica, banco, autenticação, sessão segura e layout base	Sistema local com igreja/admin, login/logout e rotas protegidas
Semana 2	Categorias, entradas, saídas, filtros, paginação e auditoria	CRUD financeiro principal funcionando com validações
Semana 3	Dashboard, relatórios, PDF, Excel e configurações	MVP funcional de ponta a ponta
Semana 4	Testes, segurança, responsividade, performance e deploy	Versão candidata para homologação

12. Critérios de Aceite Final do MVP

O MVP só deve ser considerado pronto quando todos os pontos abaixo forem validados.

12.1 Autenticação

- Criar igreja e usuário administrador.
- Fazer login com credenciais válidas.
- Bloquear login inválido com mensagem genérica.
- Fazer logout.
- Bloquear rotas privadas sem sessão.
- Expirar sessão por inatividade.

12.2 Segurança

- CSRF ativo em formulários críticos.
- Senhas armazenadas com hash.
- Queries usando prepared statements.
- Saída HTML escapada.
- Erros internos ocultos em produção.
- `.env` fora do versionamento.
- Isolamento por `igreja_id` validado.

12.3 Categorias

- Listar categorias padrão.
- Criar nova categoria.
- Editar categoria.

- Desativar categoria.
- Impedir categoria duplicada na mesma igreja.

12.4 Entradas

- Registrar dízimo.
- Registrar oferta.
- Editar entrada.
- Excluir entrada.
- Filtrar por tipo.
- Filtrar por período.
- Paginar resultados.
- Calcular total filtrado.
- Registrar auditoria nas alterações.

12.5 Saídas

- Registrar despesa.
- Editar despesa.
- Excluir despesa.
- Filtrar por categoria.
- Filtrar por período.
- Paginar resultados.
- Calcular total filtrado.
- Registrar auditoria nas alterações.

12.6 Dashboard

- Mostrar total de entradas do mês.
- Mostrar total de saídas do mês.
- Mostrar saldo do mês.
- Mostrar últimas movimentações.
- Atualizar valores após novos lançamentos.

12.7 Relatórios

- Gerar resumo por período.
- Exibir total de dízimos.
- Exibir total de ofertas.
- Exibir total de saídas.
- Exibir saldo.
- Exibir saídas por categoria.
- Exportar PDF.
- Exportar Excel.

12.8 Responsividade

- Testar em celular.
- Testar em tablet.
- Testar em desktop.

- Garantir menu usável no mobile.
- Garantir tabelas legíveis em telas pequenas.

13. Riscos Técnicos e Mitigações

Risco	Impacto	Mitigação
Não usar <code>igreja_id</code> desde o início	Retrabalho alto para virar SaaS	Incluir multi-igreja na primeira modelagem
Usar JWT sem necessidade	Complexidade e risco maior	Usar sessão segura no MVP
Falta de auditoria	Risco em sistema financeiro	Registrar logs de criação, edição e exclusão
Relatórios pesados demais	Atraso no MVP	Começar com PDF/Excel simples
Permissões complexas cedo demais	Aumenta prazo	Usar papéis simples no MVP
Interface muito sofisticada no início	Desvia foco do produto	Priorizar UX clara, responsiva e funcional
Queries sem filtro por igreja	Vazamento grave de dados	Criar padrão obrigatório de queries com <code>igreja_id</code>
Excluir categorias usadas	Quebra histórico financeiro	Usar desativação em vez de exclusão física

14. Padrões de Desenvolvimento

14.1 Código

- Usar nomes claros de variáveis, métodos e classes.
- Separar responsabilidades.
- Evitar arquivos PHP gigantes.
- Evitar SQL duplicado espalhado em vários pontos.
- Centralizar validações comuns.
- Manter mensagens de erro padronizadas.

14.2 Banco

- Usar `BIGINT UNSIGNED` para chaves principais.
- Usar `DECIMAL(12,2)` para valores financeiros.
- Usar índices nas colunas de filtro.
- Usar foreign keys.
- Evitar exclusão física em dados que afetam histórico.

14.3 Frontend

- CSS organizado por componentes.
- JavaScript simples e modular.
- Feedback visual para carregamento, sucesso e erro.
- Estados vazios em tabelas.
- Confirmação antes de excluir.

14.4 Deploy

- Ambiente de produção com `APP_ENV=production`.
 - `display_errors=Off`.
 - Logs em arquivo protegido.
 - HTTPS obrigatório.
 - Permissões corretas em diretórios.
 - Backup do banco antes de alterações relevantes.
-

15. Definição Final do MVP

O MVP revisado será considerado entregue quando a igreja conseguir:

1. acessar o sistema com segurança;
2. cadastrar suas entradas financeiras;
3. cadastrar suas despesas;
4. organizar despesas por categoria;
5. acompanhar saldo e totais no dashboard;
6. gerar relatório por período;
7. exportar PDF e Excel;
8. operar com segurança sem misturar dados entre igrejas;
9. ter registro mínimo de auditoria sobre alterações financeiras.

A implementação deve evitar funcionalidades avançadas no primeiro ciclo. O foco deve ser entregar uma base sólida, segura, profissional e pronta para evoluir.

16. Próximo Passo Técnico

A próxima etapa prática é transformar este plano em tarefas de desenvolvimento no repositório, começando por:

1. estrutura MVC;
2. configuração do ambiente;
3. criação do banco;
4. autenticação;
5. layout base;
6. CRUD financeiro.

A ordem correta é essa porque todas as funcionalidades financeiras dependem de autenticação, autorização, banco confiável e isolamento por igreja.